Université Abdelmalek Essaâdi
Ecole Nationale des Sciences Appliquées
Tanger

Système Embarqué : Horloge H24 et Thermomètre Numérique

Simulation sous Proteus et Mikro C

Réalisé par : ANDRIASAHY Tsikomia Scidelo

Année académique : 2025/2026

Table des matières

Tables des illustrations 1
I. Introduction 2
II. Étude Théorique et Matériel 2
2.1 Le Microcontrôleur PIC16F877A 2
2.2 L'Afficheur LCD (Mode 4 bits) 3
2.3 Le capteur de température LM35 5
2.4 Calcul de la conversion ADC 6
III. Réalisation et Approche Logicielle : Comparaison des Méthodes 6
3.1 Méthode 1 : Utilisation des standards des bibliothèques (conversions) 6
Méthode 2 : Approche Optimisée (Calcul Manuel) 11
IV. Application Finale : Intégration Multitâche 16
V. Conclusion Générale 19

Tables des illustrations

Figure 1 brochage du **PIC16F877A** 2
Figure 2 lecteur LCD 3
Figure 3 Tableau de brochage du lecteur LCD 3
Figure 4 connexion mode 4 bits 4
Figure 5 Mise en œuvre logicielle d'un afficheur LCD 4
Figure 6 capteur de température LM35 5
Figure 7 LM35 avec indication des broches 5
Figure 8 code pour affichage H24 avec bibliothèque 7
Figure 9 simulation Proteus H24 biblio 8
Figure 10 code pour affichage thermomètre avec bibliothèque 9
Figure 11 simulation Proteus thermomètre biblio 10
Figure 12 code pour affichage H24 avec bibliothèque 11
Figure 13 simulation sur Proteus H24 (sans bibliothèque) 1 13
Figure 14 simulation sur Proteus H24 (sans bibliothèque) 2 13
Figure 15 code pour affichage thermomètre sans utiliser bibliothèque 14
Figure 16 simulation sur Proteus thermomètre (sans bibliothèque) 1 15
Figure 17 simulation sur Proteus thermomètre (sans bibliothèque) 2 15
Figure 18 code thermo et H24 17
Figure 19 simulation sur Proteus H24 et Thermo (sans bibliothèque) 18

I. Introduction

Dans le cadre de ce travail pratique, l'objectif est de concevoir un système embarqué multitâche basé sur le microcontrôleur **PIC16F877A**. Ce système doit combiner deux fonctionnalités essentielles en instrumentation :

- La gestion du temps (Horloge H24) : Affichage de l'heure au format HH :MM : SS.
- L'acquisition de données (Thermomètre) : Mesure et affichage de la température ambiante via un capteur analogique LM35.

Une contrainte spécifique nous a été imposée : le formatage strict de l'affichage. Les valeurs doivent impérativement respecter le format "00" (ex : 01:05:09 et non 1:5:9), ce qui nécessite une gestion logicielle particulière de l'écran LCD.

II. Étude Théorique et Matériel

2.1 Le Microcontrôleur PIC16F877A

Nous utilisons le PIC16F877A pour ses capacités d'entrées/sorties et son convertisseur Analogique-Numérique (CAN/ADC) 10 bits intégré, indispensable pour lire le capteur de température.

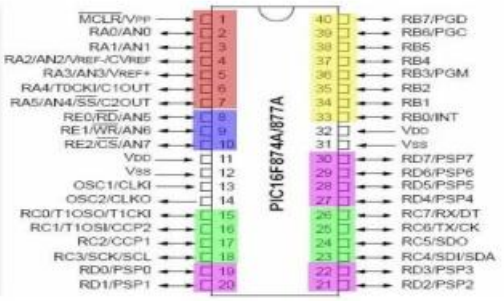

*Figure 1 brochage du **PIC16F877A***

Pour ce projet, le microcontrôleur **PIC16F877A** est cadencé par un quartz externe à une fréquence de **8 MHz**, ce qui assure une vitesse de traitement suffisante pour l'exécution fluide des routines et la gestion précise des délais.

2.2 L'Afficheur LCD (Mode 4 bits)

❖ Afficheur

LCD est l'abréviation du terme anglais **"Liquid Crystal Display"** qui signifie en français "Écran à cristaux liquides". D'où afficheur LCD.

L'afficheur LCD est en particulier une interface visuelle entre un système (projet) et l'homme (utilisateur). Son rôle est de transmettre les informations utiles d'un système à un utilisateur. Il affichera donc des données susceptibles d'être exploiter par l'utilisateur d'un système.

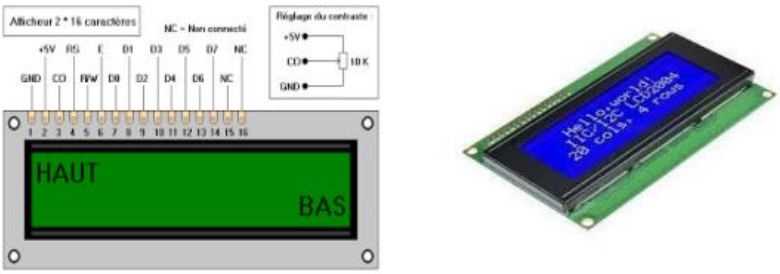

Figure 2 lecteur LCD

❖ Brochage

Le tableau ci-dessous représente bien le rôle de chaque broche et définit le brochage de l'afficheur LCD

5. 0 PIN ASSIGNMENT

No.	Symbol	Level	Function	
1	Vss	--	0V	Power Supply
2	Vdd	--	+5V	
3	V0	--	for LCD	
4	RS	H/L	Register Select: H:Data Input L:Instruction Input	
5	R/W	H/L	H--Read L--Write	
6	E	H,H-L	Enable Signal	
7	DB0	H/L	Data bus used in 8 bit transfer	
8	DB1	H/L		
9	DB2	H/L		
10	DB3	H/L		
11	DB4	H/L	Data bus for both 4 and 8 bit transfer	
12	DB5	H/L		
13	DB6	H/L		
14	DB7	H/L		
15	BLA	--	BLACKLIGHT +5V	
16	BLK	--	BLACKLIGHT 0V-	

Figure 3 Tableau de brochage du lecteur LCD

❖ Mise en œuvre matérielle d'un afficheur LCD :

En fonction du mode de commande choisis de l'afficheur LCD la mise en œuvre matérielle sera différente. Il existe deux modes de commande d'un afficheur LCD :

- commande en 4 bits,

- commande en 8 bits.

▪ **Mise en œuvre matérielle pour la commande en 4 bits (utilisé dans le TP) :**

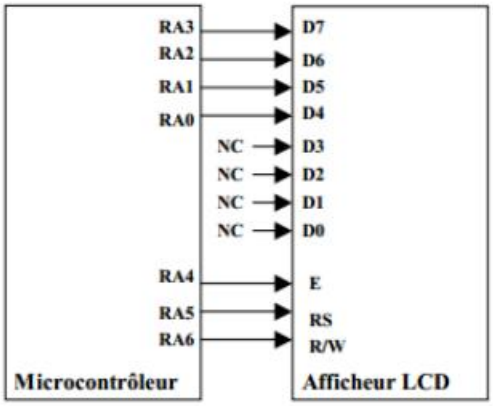

Figure 4 connexion mode 4 bits

Pour économiser les broches du microcontrôleur, nous pilotons l'écran LCD 16x2 en mode 4 bits.

Seules les broches de données D4 à D7 sont utilisées.

Les broches de contrôle sont RS (Register Select) et E (Enable).

La bibliothèque Lcd de MikroC facilite l'initialisation.

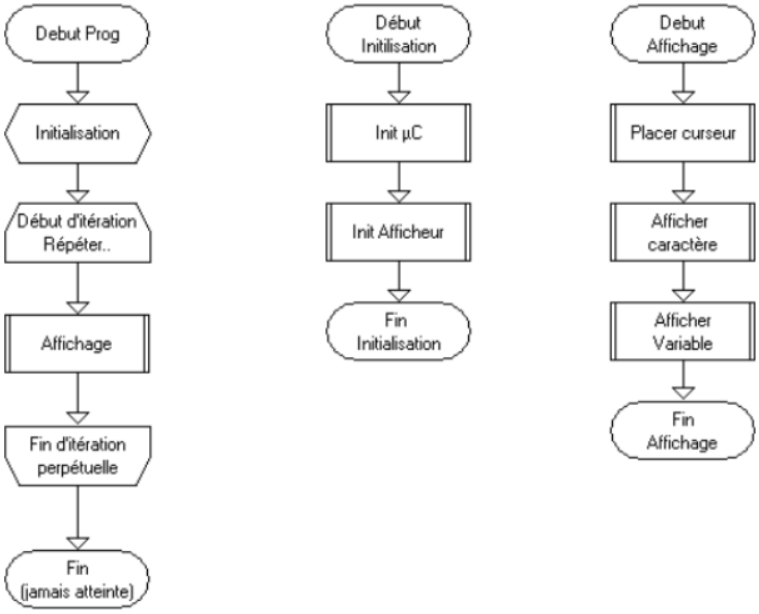

Figure 5 Mise en œuvre logicielle d'un afficheur LCD

2.3 Le capteur de température LM35

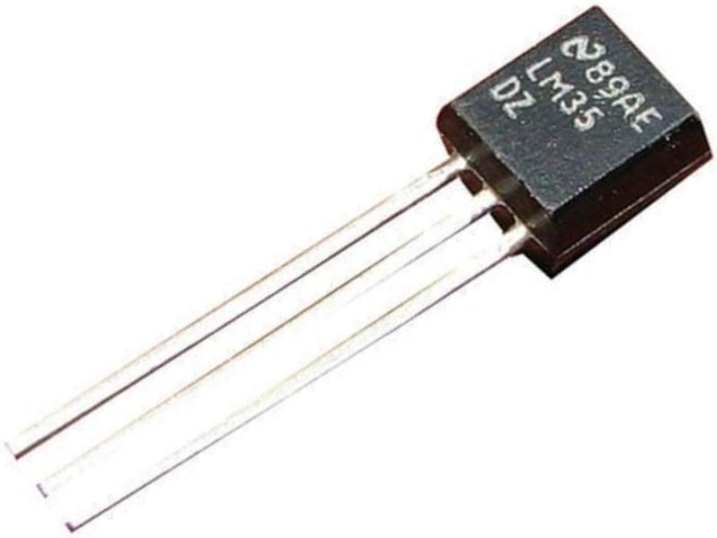

Figure 6 capteur de température LM35

Le LM35 est un capteur analogique de précision.

Fonctionnement : Il délivre une tension de sortie proportionnelle à la température en Celsius.

Échelle :$10mV/°C$.

Conversion : Pour une température de$25°C$, la sortie est de$250mV(0{,}25V)$.

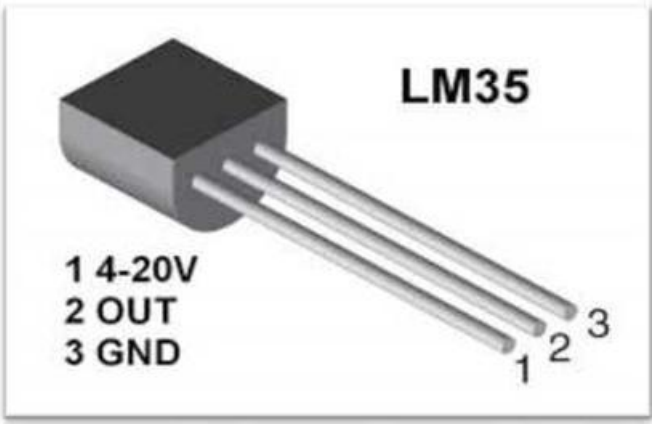

Figure 7 LM35 avec indication des broches

2.4 Calcul de la conversion ADC

Le convertisseur ADC du code PIC la tension sur 10 bits ($0 un 1023$) avec une référence de $5V$. La formule de conversion utilisée dans notre programme est:

$$Température(°C) = \frac{Valeur_{ADC} \times 5000mV}{1024} \div 10$$

III. Réalisation et Approche Logicielle : Comparaison des Méthodes

Pour ce TP, nous avons exploré deux méthodes distinctes pour gérer l'affichage et la conversion des données. Cette comparaison permet de comprendre l'impact des bibliothèques sur les ressources du microcontrôleur.

3.1 Méthode 1 : Utilisation des standards des bibliothèques (conversions)

Cette approche utilise les fonctions intégrées de MikroC comme ByteToStr et FloatToStr.

- **H24 avec code avec utilisation de la bibliothèque**

Pour cette version de l'horloge, nous avons utilisé la bibliothèque de conversion de **MikroC : ByteToStr**. Cette fonction permet de transformer une variable numérique (0-255) directement en une chaîne de caractères affichable sur l'écran LCD.

```
sbit LCD_RS at RB4_bit;
sbit LCD_EN at RB5_bit;
sbit LCD_D4 at RB0_bit;
sbit LCD_D5 at RB1_bit;
sbit LCD_D6 at RB2_bit;
sbit LCD_D7 at RB3_bit;

sbit LCD_RS_Direction at TRISB4_bit;
sbit LCD_EN_Direction at TRISB5_bit;
sbit LCD_D4_Direction at TRISB0_bit;
sbit LCD_D5_Direction at TRISB1_bit;
sbit LCD_D6_Direction at TRISB2_bit;
sbit LCD_D7_Direction at TRISB3_bit;
// --- Variables Globales ---
unsigned short heures = 0, minutes = 0, secondes = 0;
char texte_buffer[4];
// INITIALISATION
void Initialiser_Horloge() {
    TRISB = 0x00; // PORTB en sortie
    Lcd_Init();
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Cmd(_LCD_CURSOR_OFF);
    Lcd_Out(1, 1, "HORLOGE (LIB) :");
    // Affichage des séparateurs fixes
    Lcd_Chr(2, 7, ':');
    Lcd_Chr(2, 10, ':');
}
// FORMATAGE
void Afficher_Segment(unsigned short valeur, unsigned short col) {
    ByteToStr(valeur, texte_buffer);
    if (valeur < 10) {
        texte_buffer[1] = '0';
    }
    Lcd_Out(2, col, &texte_buffer[1]);
}
// GESTION DU TEMPS
void Mettre_A_Jour_Temps() {
    secondes++;
    if (secondes >= 60) {
        secondes = 0;
        minutes++;
        if (minutes >= 60) {
            minutes = 0;
            heures++;
            if (heures >= 24) {
                heures = 0;
            }
        }
    }
}
// AFFICHAGE GLOBAL

void Afficher_Heure_Complete() {
    Afficher_Segment(heures, 5);
    Afficher_Segment(minutes, 8);
    Afficher_Segment(secondes, 11);
}
// PROGRAMME PRINCIPAL
void main() {
    Initialiser_Horloge();

    while(1) {
        Afficher_Heure_Complete();
        Delay_ms(1000);
        Mettre_A_Jour_Temps();
    }
}
```

Figure 8 code pour affichage H24 avec bibliothèque

L'utilisation de ByteToStr présente une difficulté technique pour ce TP : la fonction formate les nombres sur 3 caractères avec des espaces en tête (ex : 5 devient " 5").

Pour respecter la consigne d'affichage (format 00 :00:00), nous avons implémenté un post-traitement nommé **Afficher_Segment :**

1. **Conversion** : La valeur numérique est convertie en texte par la bibliothèque.
2. **Correction du zéro** : Si le nombre est inférieur à 10, nous remplaçons manuellement l'espace par le caractère '0'.
3. **Pointeur de chaîne** : Nous utilisons un pointeur (&texte_buffer[1]) pour n'envoyer au LCD que les deux chiffres utiles, ignorant ainsi l'espace superflu du début.

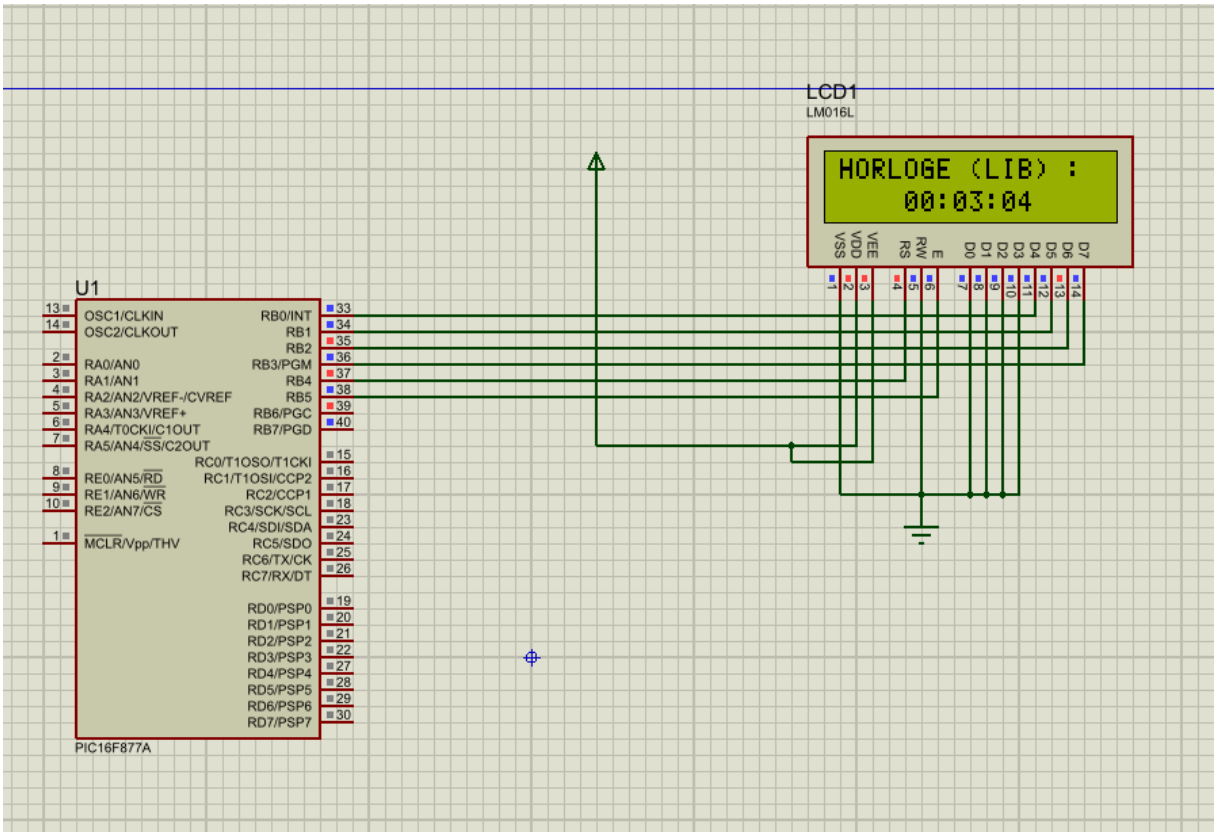

Figure 9 simulation Proteus H24 biblio

Avantages et Inconvénients

- **Avantages** : Le code est plus "standard" et utilise les outils intégrés de l'IDE, ce qui réduit les risques d'erreurs de calcul manuel.
- **Inconvénients** : Cette méthode est moins performante en termes de mémoire que la méthode manuelle (division/modulo) car elle nécessite l'inclusion de la bibliothèque Conversions et l'utilisation de buffers (tableaux de caractères) intermédiaires.

Thermomètre avec code avec utilisation de la bibliothèque

```c
#define CANAL_LM35 0
sbit LCD_RS at RB4_bit;
sbit LCD_EN at RB5_bit;
sbit LCD_D4 at RB0_bit;
sbit LCD_D5 at RB1_bit;
sbit LCD_D6 at RB2_bit;
sbit LCD_D7 at RB3_bit;

sbit LCD_RS_Direction at TRISB4_bit;
sbit LCD_EN_Direction at TRISB5_bit;
sbit LCD_D4_Direction at TRISB0_bit;
sbit LCD_D5_Direction at TRISB1_bit;
sbit LCD_D6_Direction at TRISB2_bit;
sbit LCD_D7_Direction at TRISB3_bit;

// --- Variables Globales ---
float temperature_reelle;
char texte_affichage[20];


// INITIALISATION

void Initialiser_Systeme() {
  ADCON1 = 0x82; // AN0-AN4 analogiques
  TRISA  = 0xFF; // PORTA en entrée
  TRISB  = 0x00; // PORTB en sortie

  Lcd_Init();
  Lcd_Cmd(_LCD_CLEAR);
  Lcd_Cmd(_LCD_CURSOR_OFF);

  Lcd_Out(1, 1, "Temp (Biblio) :");
}


// ACQUISITION & CONVERSION

void Traiter_Temperature() {
  unsigned int lecture_adc;

  // Lecture ADC
  lecture_adc = ADC_Read(CANAL_LM35);

  // Conversion en Celsius

  temperature_reelle = (lecture_adc * 5.0) / 1024.0 * 100.0;

  // Conversion Float vers Texte
  FloatToStr(temperature_reelle, texte_affichage);
}

// AFFICHAGE

void Afficher_Temperature() {
  short i;
  short position_point = -1;

  for(i=0; i<15; i++) {
    if (texte_affichage[i] == '.') {
      position_point = i;
      break;
    }


    if (texte_affichage[i] == '\0') break;
  }

  if (position_point != -1) {

     texte_affichage[position_point + 3] = '\0';
  }

  Lcd_Out(2, 5, "      ");
  Lcd_Out(2, 5, texte_affichage);

  Lcd_Chr_Cp(223);
  Lcd_Out_Cp("C");
}

void main() {
  Initialiser_Systeme();

  while(1) {
    Traiter_Temperature();
    Afficher_Temperature();
    Delay_ms(500);
  }
}
```

Figure 10 code pour affichage thermomètre avec bibliothèque

Bibliothèques utilisées : Pour cette version, nous avons activé les bibliothèques suivantes dans le Library Manager :

- **ADC** : Pour la lecture du capteur.
- **Conversions :** Pour la fonction FloatToStr.
- **C_String :** Indispensable pour la manipulation des chaînes de caractères (recherche du point décimal).
- **Lcd :** Pour l'affichage.

Algorithme et Fonctionnement :

- **Acquisition et Conversion** (Traiter_Temperature) : nous utilisons ici des variables de type float. La conversion de la valeur ADC (0-1023) en température se fait via la formule standard :

$$Temperature = \frac{ADC * 5{,}0}{1023} * 100$$

Le résultat est ensuite converti en chaîne de caractères via la fonction FloatToStr(temperature_reelle, texte_affichage).

- **Formatage (Afficher_Temperature)** :La fonction FloatToStr génère par défaut une chaîne avec une précision élevée, ce qui ne correspond pas au format XX.XX demandé. Pour résoudre ce problème, nous avons développé un algorithme de troncature de chaîne :
- **Recherche du séparateur** : Une boucle parcourt le tableau de caractères pour trouver l'index du point décimal (.).
- **Troncature :** Une fois le point trouvé à la position p, nous insérons le caractère de fin de chaîne (\0) à la position p + 3. Cela conserve le point + 2 décimales.

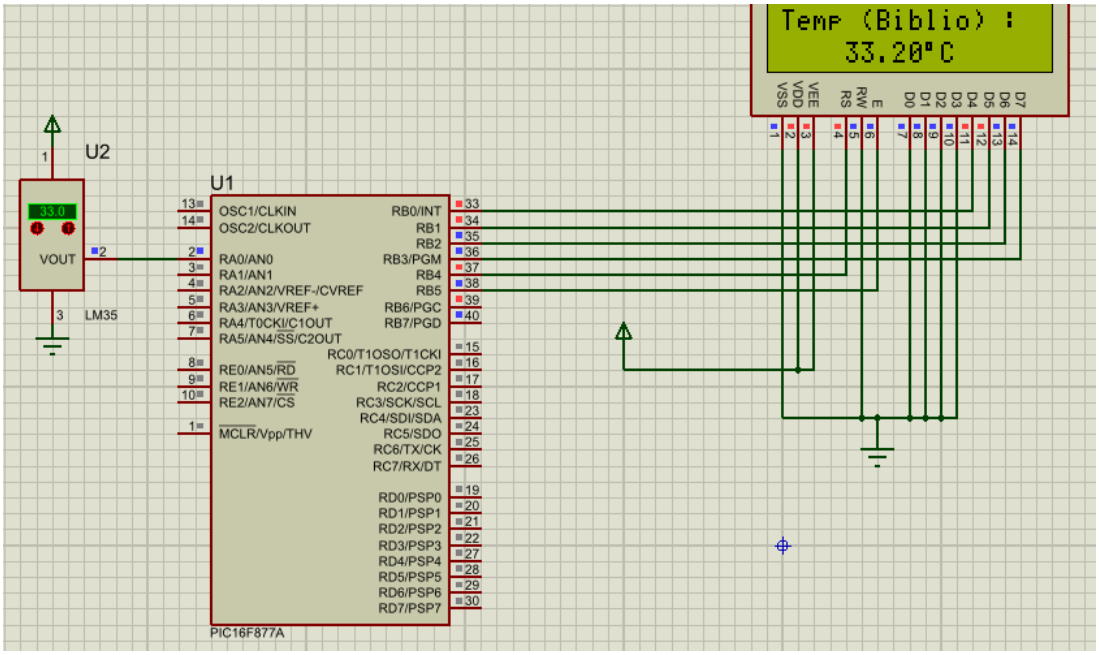

Figure 11 simulation Proteus thermomètre biblio

Méthode 2 : Approche Optimisée (Calcul Manuel)

Cette approche remplace les conversions lourdes par des opérations arithmétiques simples (division entière et modulo).

- **Avantage :** Très faible empreinte mémoire, exécution rapide, et contrôle total du formatage (ajout facile du zéro non significatif).

H24 avec code sans utilisation de la bibliothèque

```
// --- Configuration LCD ---
sbit LCD_RS at RB4_bit;
sbit LCD_EN at RB5_bit;
sbit LCD_D4 at RB0_bit;
sbit LCD_D5 at RB1_bit;
sbit LCD_D6 at RB2_bit;
sbit LCD_D7 at RB3_bit;

sbit LCD_RS_Direction at TRISB4_bit;
sbit LCD_EN_Direction at TRISB5_bit;
sbit LCD_D4_Direction at TRISB0_bit;
sbit LCD_D5_Direction at TRISB1_bit;
sbit LCD_D6_Direction at TRISB2_bit;
sbit LCD_D7_Direction at TRISB3_bit;

// --- Variables Globales ---
unsigned short h = 0, m = 0, seconde = 0;
char txt_temps[] = "00:00:00";


// INITIALISATION

void initialiser_systeme() {
    TRISB = 0x00;           // Port B en sortie
    Lcd_Init();
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Cmd(_LCD_CURSOR_OFF);
    Lcd_Out(1, 1, "HORLOGE H24");
}


//  CALCUL

void calculer_temps() {
    seconde++;
    if (seconde >= 60) {
        seconde = 0;
        m++;
        if (m >= 60) {
            m = 0;
            h++;
            if (h >= 24) {
                h = 0;
            }
        }
    }
}

// AFFICHAGE

void rafraichir_affichage() {
    // Conversion ASCII simple
    txt_temps[0] = (h / 10) + '0';
    txt_temps[1] = (h % 10) + '0';
    txt_temps[3] = (m / 10) + '0';
    txt_temps[4] = (m % 10) + '0';
    txt_temps[6] = (seconde / 10) + '0';
    txt_temps[7] = (seconde % 10) + '0';

    Lcd_Out(2, 5, txt_temps);
}


// VOID MAIN

void main() {
    initialiser_systeme();

    while(1) {
        calculer_temps();
        rafraichir_affichage();
        Delay_ms(1000);
```

Figure 12 code pour affichage H24 avec bibliothèque

Nous avons choisi de ne pas utiliser la bibliothèque Conversions de MikroC. L'objectif est de réduire l'utilisation de la mémoire Flash du PIC16F877A en effectuant nous-mêmes la décomposition des nombres en caractères affichables.

Algorithme de décomposition (Division et Modulo)

Pour afficher un nombre comme les secondes (ex: 25) sur un écran LCD, le microcontrôleur ne peut pas envoyer la valeur numérique brute. Il doit envoyer le code ASCII de chaque chiffre.

Nous utilisons donc l'arithmétique entière pour extraire chaque digit :

- **Chiffre des dizaines** : Obtenu par une division entière par 10 (valeur / 10).
- **Chiffre des unités** : Obtenu par l'opération modulo 10 (valeur % 10).

Conversion en code ASCII

Une fois les chiffres extraits (ex: 2 et 5), nous devons les convertir en caractères. En suivant la table ASCII, le chiffre '0' correspond à la valeur décimale 48. L'astuce consiste donc à ajouter 48 (ou le caractère '0') au résultat de nos calculs :

- Caractère = (Valeur / 10) + 48

Gestion du formatage strict (Format 00)

L'avantage majeur de cette méthode manuelle est qu'elle gère nativement le formatage imposé.

- Si la valeur est 5, le calcul des dizaines (5 / 10) donne 0.
- L'écran affichera donc 0 puis 5, soit 05.

Contrairement aux bibliothèques standards qui remplacent souvent le zéro par un espace, notre routine garantit un affichage constant à deux chiffres.

Avantages de cette méthode

- **Légèreté** : Occupation minimale de la mémoire programme (Flash).
- **Vitesse** : Pas de fonctions de conversion complexes à exécuter.
- **Précision du formatage** : Contrôle total sur l'affichage des zéros non significatifs.

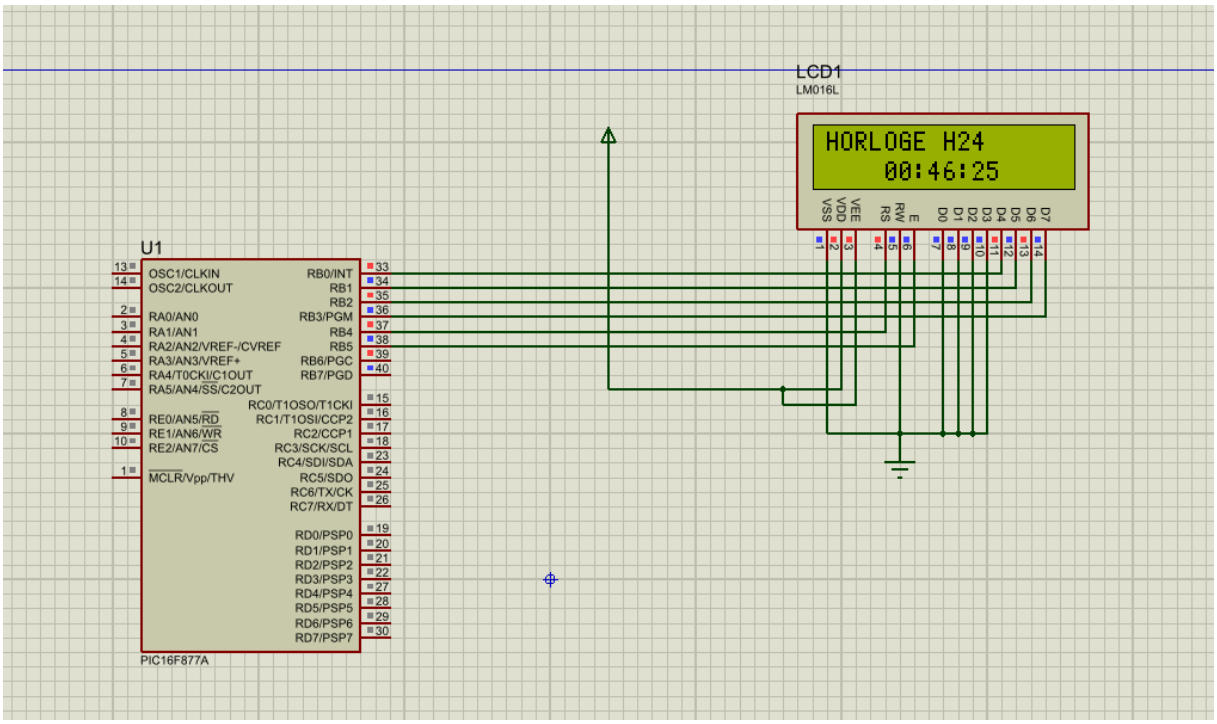

Figure 13 simulation sur Proteus H24 (sans bibliothèque) 1

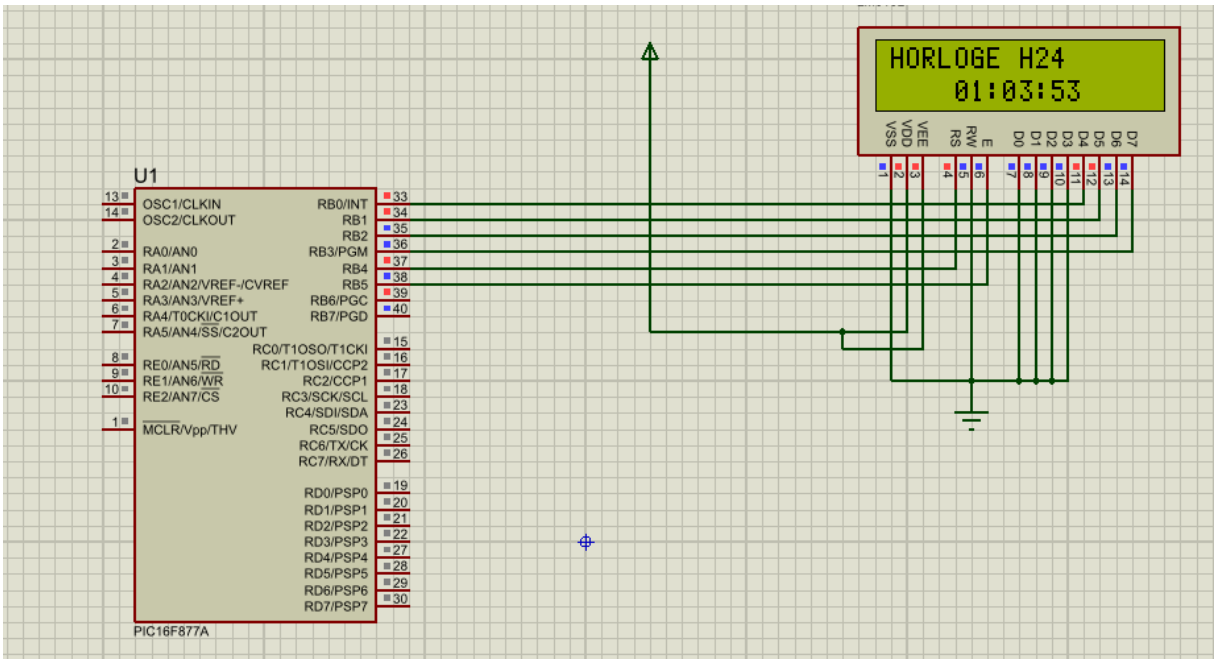

Figure 14 simulation sur Proteus H24 (sans bibliothèque) 2

Thermomètre avec code sans utilisation de la bibliothèque

```
#define CANAL_LM35 0
// --- Connexions LCD ---
sbit LCD_RS at RB4_bit;
sbit LCD_EN at RB5_bit;
sbit LCD_D4 at RB0_bit;
sbit LCD_D5 at RB1_bit;
sbit LCD_D6 at RB2_bit;
sbit LCD_D7 at RB3_bit;

sbit LCD_RS_Direction at TRISB4_bit;
sbit LCD_EN_Direction at TRISB5_bit;
sbit LCD_D4_Direction at TRISB0_bit;
sbit LCD_D5_Direction at TRISB1_bit;
sbit LCD_D6_Direction at TRISB2_bit;
sbit LCD_D7_Direction at TRISB3_bit;
// --- Variables Globales ---
unsigned int temp_calcul;
// INITIALISATION
void Initialiser_Systeme() {
  ADCON1 = 0x82;
  TRISA  = 0xFF; // PORTA en entrée (Capteur)
  TRISB  = 0x00; // PORTB en sortie (LCD)

  Lcd_Init();
  Lcd_Cmd(_LCD_CLEAR);
  Lcd_Cmd(_LCD_CURSOR_OFF);

  Lcd_Out(1, 1, "Temp LM35 :");
}
// D'ACQUISITION & CALCUL
void Traiter_Capteur() {
  unsigned int lecture_adc;
  unsigned long calcul_intermediaire;
  lecture_adc = ADC_Read(CANAL_LM35);

  calcul_intermediaire = (unsigned long)lecture_adc * 50000;
  temp_calcul = (unsigned int)(calcul_intermediaire / 1024);
}
// ROUTINE D'AFFICHAGE
void Afficher_Resultat() {
  char dizaine, unite, decimale1, decimale2;

  // Si la température dépasse 99.99°C, on bloque
  if (temp_calcul > 9999) temp_calcul = 9999;

  dizaine   = (temp_calcul / 1000) + 48;
  unite     = ((temp_calcul / 100) % 10) + 48;
  decimale1 = ((temp_calcul / 10) % 10) + 48;
  decimale2 = (temp_calcul % 10) + 48;

  Lcd_Chr(2, 5, dizaine);
  Lcd_Chr(2, 6, unite);
  Lcd_Chr(2, 7, '.');
  Lcd_Chr(2, 8, decimale1);
  Lcd_Chr(2, 9, decimale2);

  Lcd_Chr(2, 11, 223); // Symbole Degré
  Lcd_Out(2, 12, "C");
}

// PROGRAMME PRINCIPAL
void main() {
  Initialiser_Systeme();

  while(1) {
    Traiter_Capteur();
    Afficher_Resultat();
    Delay_ms(500);
  }
}
```

Figure 15 code pour affichage thermomètre sans utiliser bibliothèque

Le processus se déroule en trois étapes clés :

- **Acquisition Analogique (ADC_Read)** : Le convertisseur ADC (Analog-to-Digital Converter) du PIC lit la tension sur le canal AN0. La valeur retournée est comprise entre 0 et 1023 (pour 0 à 5V).
- **Conversion Mathématique Optimisée**
- **Affichage Caractère par Caractère** : En ajoutant 48 (code ASCII de '0') à chaque résultat, nous obtenons les caractères affichables sur l'LCD.

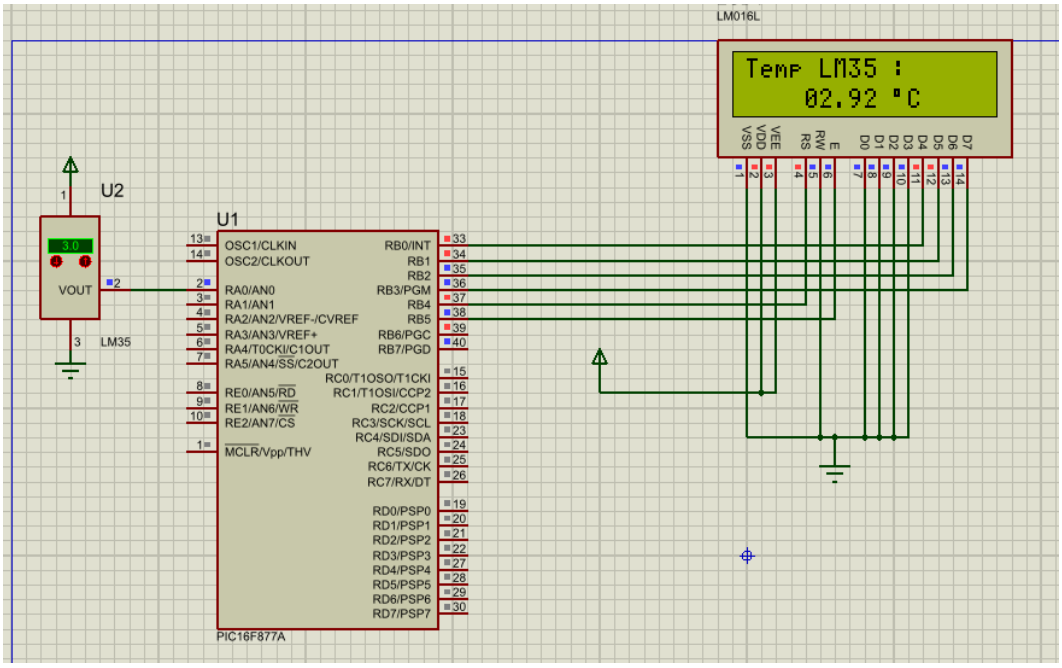

Figure 16 simulation sur Proteus thermomètre (sans bibliothèque) 1

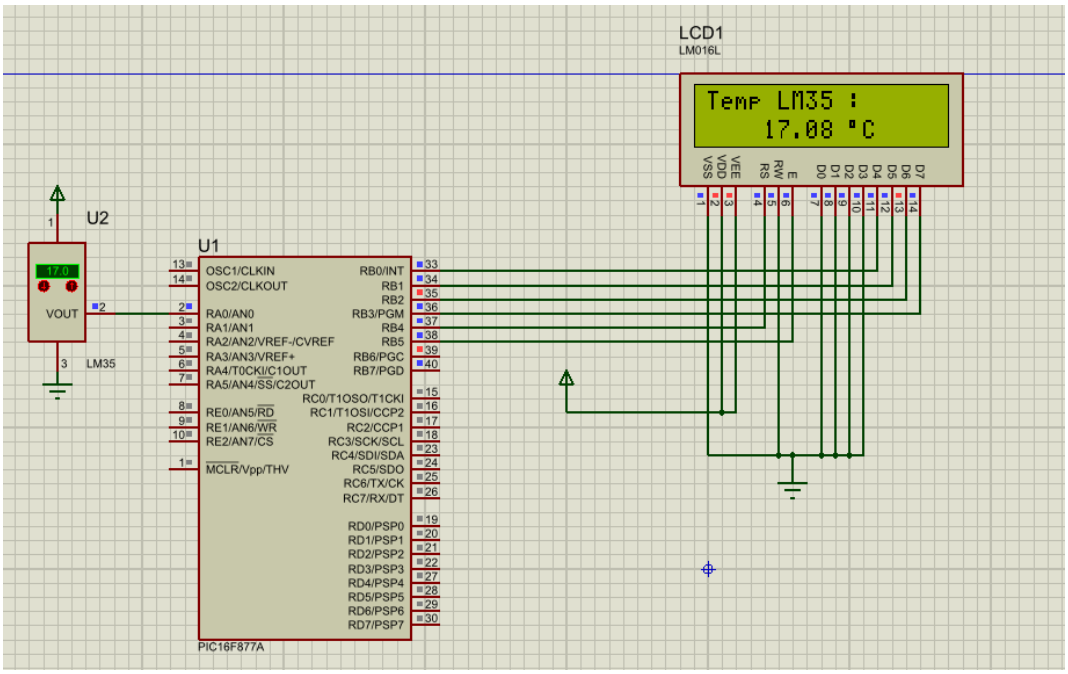

Figure 17 simulation sur Proteus thermomètre (sans bibliothèque) 2

Lors de la simulation sous Proteus, nous avons pu vérifier que l'affichage réagit instantanément aux variations du capteur LM35 virtuel, affichant une température précise au centième de degré validant ainsi notre algorithme de conversion optimisé.

IV. Application Finale : Intégration Multitâche

L'application finale combine les deux modules précédents (Horloge et Thermomètre) dans une structure logicielle unifiée. Le défi technique principal consiste à gérer l'affichage dynamique sur les deux lignes du LCD sans que l'un des processus ne bloque l'autre.

Le programme utilise une architecture modulaire composée de quatre routines principales :

- **Initialiser_Systeme** : Configure les registres ADC, les directions des ports (TRIS) et l'interface LCD.
- **Gerer_Thermometre** : Effectue l'acquisition analogique et formate le résultat au format **XX.XX**.
- **Gerer_Horloge** : Gère la logique de comptage du temps et l'affichage au format **HH:MM:SS**.
- **Afficher_Horloge_Unite** : Une routine utilitaire de bas niveau pour le formatage des chiffres avec zéro à gauche.

Optimisation du Formatage (Format XX.XX et HH:MM:SS)

- **Horloge** : La routine décompose chaque segment (H, M, S) pour forcer l'affichage d'un zéro si la valeur est inférieure à 10.
- **Température** : Le calcul est effectué avec une précision au centième. Nous multiplions la valeur ADC par un facteur de 50000 (en utilisant le type unsigned long pour éviter les dépassements de capacité) avant la division par 1024. Cela nous permet d'extraire deux décimales après la virgule sans jamais utiliser de nombres flottants, garantissant ainsi une exécution très rapide du processeur.

```c
sbit LCD_RS at RB4_bit;
sbit LCD_EN at RB5_bit;
sbit LCD_D4 at RB0_bit;
sbit LCD_D5 at RB1_bit;
sbit LCD_D6 at RB2_bit;
sbit LCD_D7 at RB3_bit;

sbit LCD_RS_Direction at TRISB4_bit;
sbit LCD_EN_Direction at TRISB5_bit;
sbit LCD_D4_Direction at TRISB0_bit;
sbit LCD_D5_Direction at TRISB1_bit;
sbit LCD_D6_Direction at TRISB2_bit;
sbit LCD_D7_Direction at TRISB3_bit;
// --- Variables Globales ---
unsigned short h = 0, m = 0, seconde = 0; // Variables Horloge
unsigned int temp_centi;
// INITIALISATION
void Initialiser_Systeme() {
    ADCON1 = 0x80;    // RA0 en analogique, Vref = 5V
    TRISA = 0xFF;     // PORTA en entrée
    TRISB = 0x00;     // PORTB en sortie
    Lcd_Init();
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Cmd(_LCD_CURSOR_OFF);
// Affichage des éléments statiques (fixes)
    Lcd_Out(1, 1, "Temp:      C");
    Lcd_Chr(1, 11, 223); // Symbole Degré
    Lcd_Out(2, 7, ":");
    Lcd_Out(2, 10, ":");
}
// FORMATAGE "00" (Horloge)
void Afficher_Horloge_Unite(unsigned short val, unsigned short col) {
    Lcd_Chr(2, col, (val / 10) + '0');     // Chiffre des dizaines
    Lcd_Chr(2, col + 1, (val % 10) + '0'); // Chiffre des unités
}
// GESTION THERMOMÈTRE (XX.XX)
void Gerer_Thermometre() {
    unsigned long adc_val;
    // Lecture et calcul (Format XX.XX -> x100)
    adc_val = ADC_Read(0);
    temp_centi = (unsigned int)((adc_val * 50000ul) / 1024);
    // Décomposition et affichage manuel
    Lcd_Chr(1, 7, (temp_centi / 1000) + '0');        // Dizaines
    Lcd_Chr(1, 8, ((temp_centi / 100) % 10) + '0');  // Unités
    Lcd_Chr(1, 9, '.');                              // Point
    Lcd_Chr(1, 10, ((temp_centi / 10) % 10) + '0');  // Dixièmes
    Lcd_Chr(1, 11, (temp_centi % 10) + '0');         // Centièmes

    Lcd_Out(1, 13, "C");
}
// GESTION HORLOGE (HH:MM:SS)
void Gerer_Horloge() {
    seconde++;
    if (seconde >= 60) {
        seconde = 0;
        m++;
        if (m >= 60) {
            m = 0;
            h++;
            if (h >= 24) {
                h = 0;
            }
        }
    }
    Afficher_Horloge_Unite(h, 5);
    Afficher_Horloge_Unite(m, 8);
    Afficher_Horloge_Unite(seconde, 11);
}
// PROGRAMME PRINCIPAL
void main() {
    Initialiser_Systeme();

    while(1) {
        Gerer_Thermometre();
        Gerer_Horloge();

        Delay_ms(1000);
    }
}
```

Figure 18 code thermo et H24

Simulation et Validation

Le projet a été validé sous Proteus ISIS. Nous avons pu observer :

- Le défilement régulier des secondes sur la ligne inférieure.
- La mise à jour de la température au centième de degré sur la ligne supérieure lors de la manipulation du capteur LM35.
- Le respect parfait des alignements et des formats (zéros présents pour les heures/minutes/secondes simples).

Cette application démontre la capacité du PIC16F877A à gérer simultanément de l'acquisition analogique et du traitement temporel tout en pilotant une interface utilisateur visuelle.

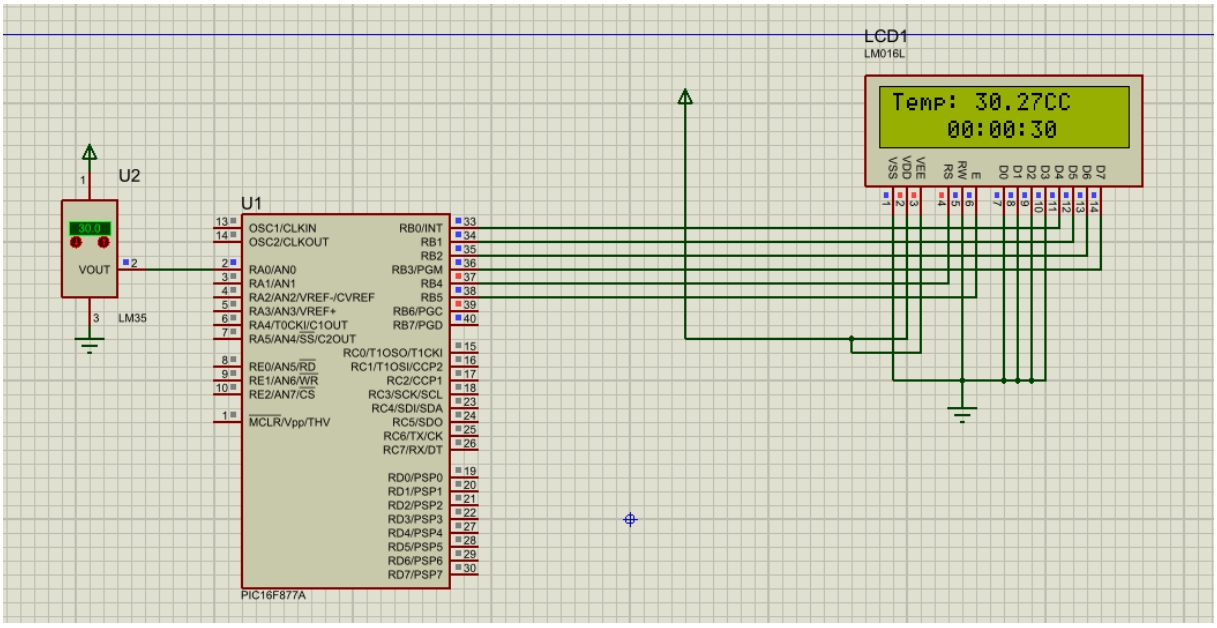

Figure 19 simulation sur Proteus H24 et Thermo (sans bibliothèque)

V. Conclusion Générale

La réalisation de ce TP a permis de mettre en pratique les concepts fondamentaux de la programmation des microcontrôleurs PIC en langage C via l'IDE MikroC. À travers le développement de cette horloge et de ce thermomètre numérique, nous avons pu approfondir la gestion des périphériques internes tels que le convertisseur analogique-numérique (ADC) et les ports d'entrées/sorties pour le pilotage d'un afficheur LCD.

L'aspect le plus formateur a été la mise en œuvre d'une programmation optimisée. En choisissant une approche "manuelle" pour le formatage des données au lieu de dépendre exclusivement de bibliothèques lourdes, nous avons optimisé l'utilisation de la mémoire Flash et de la RAM du microcontrôleur. Cette rigueur algorithmique nous a permis de respecter scrupuleusement les contraintes d'affichage imposées (format XX.XX et HH:MM:SS), garantissant ainsi un système fiable et professionnel.

En conclusion, ce travail souligne l'importance de l'optimisation logicielle dans les systèmes embarqués et constitue une base solide pour des projets plus complexes impliquant des interruptions.